

Spring Bean Lifecycle — Quick Revision

1. What is Bean Lifecycle?

In normal Java, you create and manage objects yourself. In Spring, you tell Spring which classes to manage (using `@Component` or `@Bean`), and Spring takes over that object's full journey.

2. Simple Definition

Bean lifecycle = the complete journey of a Spring-managed object, from the moment Spring discovers it till the moment Spring destroys it. A Spring-managed object is called a **bean**.

3. Complete Bean Lifecycle Flow

Order: Container starts → read config → create BeanDefinition → instantiate object → inject dependencies → Aware callbacks → init callbacks → bean ready → use bean → destroy callbacks → bean removed.

4. Step 0: Spring Container Starts

When you pass `AppConfig.class` to the container, you're not passing an object — you're passing class metadata. Spring uses reflection to read annotations and understand the app structure.

5. Step 1: Bean Definition is Read

Before creating any object, Spring collects metadata (name, class, scope, dependencies, init/destroy methods) called a **BeanDefinition**. No object exists yet — just info about it. This can come from annotations, `@Bean` methods, or XML.

6. Step 2: Bean Object is Instantiated

Spring calls the constructor to actually create the object. Remember: **Instantiation** (object created) ≠ **Initialization** (object prepared after creation) — they are different phases.

7. Step 3: Dependencies are Injected

Spring supplies required dependencies via **Constructor Injection** (passed while creating object), **Setter Injection** (object created first, then setter called), or **Field Injection** (object created first, then field set via reflection). Constructor injection is preferred for clean/testable code.

8. Step 4: Aware Interfaces are Called

If a bean needs container info (like its own name or the `ApplicationContext`), Spring calls special "Aware" callback methods (e.g. `BeanNameAware`, `ApplicationContextAware`)

automatically. Rarely used in normal business logic — mostly for framework/infra code.

9. Step 5: Initialization Callbacks

After object creation + DI + Aware calls, Spring lets the bean run startup logic. Three ways: `@PostConstruct` (recommended), `InitializingBean.afterPropertiesSet()` (coupled to Spring), or custom `initMethod` in `@Bean`. If all used together, order is: `@PostConstruct` → `InitializingBean` → custom `initMethod`.

10. Step 6: Bean is Ready to Use

After initialization, the bean is fully ready. For singleton beans, Spring stores and reuses the same instance every time it's needed.

11. Step 7: Destruction Callbacks

When container shuts down, Spring lets beans clean up (close connections, stop threads, etc.). Three ways: `@PreDestroy` (recommended), `DisposableBean.destroy()`, or custom `destroyMethod`. Order if combined: `@PreDestroy` → `DisposableBean` → custom `destroyMethod`.

12. Full Lifecycle Summary

Container starts → read config → BeanDefinition → instantiate → inject deps → Aware calls → init callbacks → ready → used → destroy callbacks → removed. This is the full 11-step flow to remember.

13. Singleton Bean Lifecycle

Default scope. Spring creates **only one object**, manages its full lifecycle: create → inject → init → use → destroy. Spring creates it, stores it, reuses it, destroys it.

14. Prototype Bean Lifecycle

A **new object every time** the bean is requested (`getBean()` called). Spring creates it, injects deps, runs init callbacks — but does NOT track it for automatic destruction after handing it over.

15. Prototype Destruction Does Not Happen Automatically

`@PreDestroy` won't run for prototype beans on `context.close()`. Reason: Spring doesn't keep references to every prototype instance (would cause memory leaks / block garbage collection). If a prototype bean holds heavy resources, clean up manually.

16. Singleton Depends on Prototype

If a singleton (e.g. `OrderService`) injects a prototype bean (e.g. `PaymentSession`) via constructor, Spring creates the prototype **only once** — at singleton creation time — not

fresh on every use. To get a new prototype instance every time, use `ObjectProvider`, `Provider`, lookup method injection, or `applicationContext.getBean()`.

17. @Lazy and Bean Lifecycle

By default, singleton beans are created **eagerly** (immediately at startup). Marking a bean `@Lazy` delays actual object creation, DI, and initialization until it's first requested. BeanDefinition is still read at startup though.

18. @PostConstruct and Circular Dependency

If two beans (A and B) depend on each other via **constructor injection**, Spring can't create either first — this fails. Fix: inject one via constructor, and set the other reference inside `@PostConstruct` after both objects exist.

20. Final Revision Summary

Full phase order: BeanDefinition reading → object creation → DI → Aware callbacks → BeanPostProcessor (before init) → init callbacks → BeanPostProcessor (after init) → bean ready → destruction callbacks.

- **Singleton:** full lifecycle managed by Spring (create → destroy).
- **Prototype:** Spring creates, injects, initializes, hands over — no auto-destroy.
- **@Lazy singleton:** BeanDefinition read at startup, object creation delayed till needed.
- **@PostConstruct:** runs logic right after DI, part of init phase.

21. One-Line Takeaway

Spring Bean Lifecycle is not just object creation — it's the full process of Spring **discovering, creating, preparing, managing, and destroying** a bean.